

Humanization Improvements Summary

Project: Context-Aware Churn Prediction Model

Date: January 23, 2026

Reference: Patterns derived from `bank-marketing-propensity-model`

Overview

This document summarizes all humanization improvements applied to the context-aware churn model repository to enhance code readability, maintainability, and developer experience.

Table of Contents

1. [Configuration Management](#)
 2. [Logging Infrastructure](#)
 3. [Documentation & Docstrings](#)
 4. [Type Hints](#)
 5. [Test Organization](#)
 6. [Script Enhancements](#)
 7. [Code Style & Readability](#)
 8. [Files Modified](#)
-

1. Configuration Management

Before

- Basic configuration with minimal structure
- No visual separators or grouping
- Missing directory auto-creation
- Limited type hints

After

- **Visual separators** with `=====` for clear section delineation:
- Project Paths
- Data Configuration
- Model Configuration
- NLP Configuration
- Feature Engineering
- API Configuration
- Logging Configuration
- Output Configuration

- **Automatic directory creation** on import:

```
python
for directory in [RAW_DATA_DIR, PROCESSED_DATA_DIR, MODELS_DIR, LOGS_DIR, OUTPUT_DIR]:
    directory.mkdir(parents=True, exist_ok=True)
```

- **Typed dictionaries** with `Dict[str, Any]` for all config objects

- **Environment variable support** for production deployment:

```
python
"host": os.getenv("API_HOST", "0.0.0.0"),
"port": int(os.getenv("API_PORT", "8000")),
```

- **Inline comments** explaining non-obvious settings:

```
python
"class_weight": "balanced", # helps with class imbalance
```

2. Logging Infrastructure

Before

- Mix of `print()` statements and basic logging
- No dual handler setup (file + console)
- Inconsistent formatting

After

Module-Level Logging Setup

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger(__name__)
```

Dual Handlers (File + Console)

```
handlers=[
    logging.FileHandler(LOGGING_CONFIG["log_file"]),
    logging.StreamHandler(sys.stdout),
]
```

Structured Log Output

```
logger.info("=" * 60)
logger.info("TRAINING CHURN PREDICTION MODEL")
logger.info("=" * 60)
```

Enhanced Logger Utility

- Support for both standard `logging` and `loguru`
- Graceful degradation if `loguru` not installed

- Configurable log rotation and retention

3. Documentation & Docstrings

Module-Level Docstrings

Every module now starts with a comprehensive docstring:

```
"""
Explainable churn prediction model with SHAP interpretability.

This module provides the core ExplainableChurnModel class that handles:
- Model initialization (Random Forest or Gradient Boosting)
- Training with cross-validation and performance metrics
- SHAP-based prediction explanations
- Feature importance extraction
- Model persistence (save/load)
"""
```

Class Docstrings

```
class ExplainableChurnModel:
    """
    Churn prediction model with SHAP explainability.

    Supports Random Forest and Gradient Boosting classifiers with
    built-in SHAP integration for model interpretability.

    Attributes:
        model_type: Type of model ('random_forest' or 'gradient_boosting')
        random_state: Random seed for reproducibility
        model: Underlying sklearn classifier
        explainer: SHAP TreeExplainer instance
        feature_names: List of feature column names
    """
```

Function Docstrings (Google Style)

```
def train(
    self,
    X: pd.DataFrame,
    y: pd.Series,
    test_size: float = 0.2,
) -> Dict[str, Any]:
    """
    Train the churn prediction model.

    Args:
        X: Feature DataFrame
        y: Target Series (0=retained, 1=churned)
        test_size: Fraction of data for testing

    Returns:
        Dictionary containing training metrics and test data
    """
```

4. Type Hints

Before

```
def train(self, X, y, test_size=0.2):
```

After

```
def train(  
    self,  
    X: pd.DataFrame,  
    y: pd.Series,  
    test_size: float = 0.2,  
    cv_folds: int = 5,  
    run_cv: bool = True  
) -> Dict[str, Any]:
```

Type Hints Applied

Module	Functions with Type Hints
churn_model.py	9 functions
preprocessor.py	5 functions
data_generator.py	4 functions
main.py (API)	5 functions
logger.py	4 functions

Complex Return Types

```
def prepare_features(  
    self,  
    users: pd.DataFrame,  
    interactions: pd.DataFrame,  
    include_embeddings: bool = False  
) -> Tuple[pd.DataFrame, pd.Series]:
```

5. Test Organization

Before

- Flat function-based tests
- Basic fixtures
- Generic test names

After

Class-Based Organization

```
class TestModelInitialization:
    """Tests for model initialization."""

class TestModelTraining:
    """Tests for model training functionality."""

class TestModelPrediction:
    """Tests for model prediction functionality."""

class TestFeatureImportance:
    """Tests for feature importance extraction."""

class TestExplainPrediction:
    """Tests for SHAP-based prediction explanations."""
```

Descriptive Test Names

```
def test_creates_random_forest_model(self):
    """Random forest model should be created with correct type."""

def test_raises_error_for_unknown_model_type(self):
    """Unknown model type should raise ValueError."""

def test_train_accuracy_is_reasonable(self, sample_data):
    """Trained model should have reasonable accuracy."""
```

Proper Fixtures

```
@pytest.fixture
def trained_model_with_data(self):
    """Create trained model with test data."""
    np.random.seed(42)
    # ... setup code ...
    return model, X
```

6. Script Enhancements

Before

- No command-line arguments
- Basic print statements
- No help text

After

Full Argparse Integration

```
parser = argparse.ArgumentParser(
    description="Train churn prediction model",
    formatter_class=argparse.RawDescriptionHelpFormatter,
    epilog="""
Examples:
python scripts/train_model.py
python scripts/train_model.py --model-type gradient_boosting
python scripts/train_model.py --cv-folds 10 --embeddings
    """
)

parser.add_argument(
    "--model-type",
    default="random_forest",
    choices=["random_forest", "gradient_boosting"],
    help="Type of model to train (default: random_forest)"
)
```

Structured Pipeline Output

```
logger.info("=" * 60)
logger.info("TRAINING COMPLETE")
logger.info("=" * 60)
logger.info(f" Test Accuracy: {results['test_accuracy']:.4f}")
logger.info(f" ROC-AUC Score: {results['roc_auc']:.4f}")
logger.info("\n✓ Training pipeline finished successfully!")
```

7. Code Style & Readability

Visual Separators

```
# =====
# Project Paths
# =====
```

Logical Grouping with Comments

```
# -----
# aggregate basic features per user
# -----
user_features = interactions.groupby("user_id").agg({...})

# -----
# calculate recency and frequency
# -----
user_features["recency_days"] = (...)
```

Consistent Naming Conventions

Type	Convention	Examples
Functions	verb_noun	load_data , train_model , get_feature_importance
Classes	PascalCase	ExplainableChurnModel , ChurnPreprocessor
Constants	UPPER_SNAKE	MODEL_CONFIG , LOG- GING_CONFIG
Variables	snake_case	feature_names , test_size

Graceful Dependency Handling

```
try:
    import shap
    SHAP_AVAILABLE = True
except ImportError:
    SHAP_AVAILABLE = False

# Later in code:
if not SHAP_AVAILABLE:
    logger.warning("SHAP not available. Install with: pip install shap")
    return
```

8. Files Modified

File	Changes
<code>src/config.py</code>	Visual separators, typed dicts, env vars, auto-create dirs
<code>src/models/churn_model.py</code>	Logging, type hints, docstrings, SHAP availability check
<code>src/data/preprocessor.py</code>	Logging, type hints, docstrings, structured sections
<code>src/data/data_generator.py</code>	Logging, type hints, docstrings, argparse
<code>src/api/main.py</code>	Logging, helper functions, Pydantic docstrings
<code>src/utils/logger.py</code>	Dual handlers, loguru support, configurable setup
<code>tests/test_model.py</code>	Class-based organization, descriptive names, fixtures
<code>tests/test_api.py</code>	Class-based organization, descriptive names
<code>scripts/train_model.py</code>	Argparse, logging, structured output
<code>scripts/generate_data.py</code>	Argparse, logging, structured output

Checklist Completion

Pattern	Applied
Directory structure organization	✓
Comprehensive README	✓ (already good)
Module-level docstrings	✓
Centralized config.py	✓
Type hints on all functions	✓
Replace print with logging	✓
Descriptive function names (verb_noun)	✓
Inline comments for complex logic	✓
Test class organization	✓
Visual separators in config/logs	✓
Command-line arguments with argparse	✓
Graceful optional dependency handling	✓
Consistent naming conventions	✓
Docstrings on test functions	✓
Fixtures for test data	✓
Log section headers	✓
Environment variable support	✓

Next Steps

1. **Add more tests** - Increase test coverage for data preprocessing
2. **Add hyperparameter tuning** - Implement `--tune` flag in training script
3. **Add model versioning** - Track model versions with MLflow or similar
4. **Add CI integration** - Update GitHub Actions for new test structure
5. **Add type checking** - Run mypy for static type validation

End of Document